PairCover-Q: presentation speech

About 10 minutes. Updated 28 Oct 2026 with the real IBM hardware run. Blue boxes = what to say. Yellow boxes = what to click or show.

---

1. Opening and the problem (1 min)

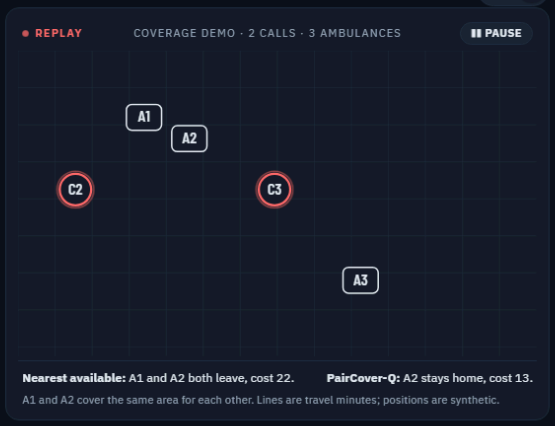
COVERAGE-AWARE AMBULANCE DISPATCH · QUBO + QAOA

Send the right ambulance. Keep the city covered.

When emergencies arrive together, sending the nearest ambulance to each one can leave a neighbourhood with **no backup at all**. PairCover-Q weighs response time against the coverage left behind, writes that trade-off in the form quantum computers accept, and solves it three ways: **the usual rule, an exact search, and a quantum algorithm**.

► See the decision in one click | Create your own emergency

✓ Every result checked against an exact solver ✓ Synthetic city, clearly labelled
✓ No quantum advantage claimed



Nearest available: A1 and A2 both leave, cost 22. **PairCover-Q:** A2 stays home, cost 13.
A1 and A2 cover the same area for each other. Lines are travel minutes; positions are synthetic.

HOW IT WORKS

Calls 1

Emergencies arrive together

Several calls, a few free ambulances, and stations that cover the same area for each other. Send both partners away and that area has no one left.

QUBO 2

One yes/no per choice

"Does ambulance A go to call C?" becomes one binary variable, one qubit. Losing a backup pair is a penalty between exactly two variables, which is the shape quantum optimisers take natively.

Verify 3

Three solvers, one answer key

Nearest-available, exhaustive search and QAOA solve the same problem. Every quantum result is graded against the exact answer and against random guessing.

WHAT WE FOUND

22 → 13

On the coverage demo, keeping one ambulance home costs **1 more weighted minute** but removes a coverage penalty of 10, cutting total cost by 9.

Computed live by the solvers on this page load

−2.15 min

Mean response time at 5 calls per hour in a simulated city (7 days × 30 runs) against nearest-available, 95% CI −2.67 to −1.63. Planning simultaneous calls together explains this: batching alone gets −2.18. The coverage term's own effect is slightly less uncovered demand, with no significant change in late arrivals.

Source: rolling-simulation report

19 of 23

Simulation-derived problems (4–12 qubits) where depth-3 QAOA put more probability on the optimum than random guessing among valid plans. Simulated, noiseless; noise erodes the gain, and the exact solver is still faster at these sizes.

Source: QAOA benchmark report

Honest by design The city, calls and ambulances are synthetic test data. QAOA runs on a noiseless simulator. At these sizes an ordinary computer solves the problem instantly, so we claim no quantum advantage: we show where quantum fits, and what it would take.

Show the Home page. Let the animation play.

Good evening sir, we are Team Evric, and our project is PairCover-Q. It is about ambulance dispatch.

Normally the dispatcher sends the nearest free ambulance to each emergency. That works for one call. But when several calls come at the same time, sending the nearest ambulance to every call can leave a whole area with no backup.

In this animation, A1 and A2 cover the same area for each other. Nearest-available sends both of them, so that area is empty, and the cost is 22. Our method keeps A2 at home and sends A3 to the second call. One call waits one minute longer, but the area stays protected, and the cost is 13.

---

2. How we model it (2 min)

Scroll to "How it works" on the Home page (three cards).

We turn this into a math problem. For every choice "does ambulance A go to call C", we make one yes/no variable. That is one qubit.

Our cost has three parts: priority-weighted response time, a penalty if a call is left unserved, and a pairwise penalty that switches on only when two backup partners both leave. That last term is a product of exactly two variables, so the whole problem is a QUBO, which is the form that quantum optimisers accept. We need no extra qubits for the coverage rule.

Then we solve the same problem three ways: the nearest-ambulance rule, an exact brute-force search, and QAOA in Qiskit, on a simulator and on a real IBM quantum computer. Every quantum answer is checked against the exact answer and against random guessing.

---

3. Live demo: the dispatch console (2 min)

PairCover-Q

Home | Try it | Quantum | About

TRY IT

Dispatch console

Calls on the left, the response zone in the middle, the decision on the right. Run two methods and the console explains what changed.

Coverage demo · 6 qubits

2 open calls **3** available units **1** backup pair (A1+A2) — dispatch cost of the plan shown

✓ Calls arrive | 2 Choose a method | 3 Run it | 4 Compare & see why

emergency call · ambulance dispatched · held in reserve / backup kept · coverage at risk · quantum

No decision yet. 2 calls are waiting and 3 ambulances are free. Choose a method on the right and press **Run**, or compare two methods in one click.

▸ Compare nearest vs exact

CALLS WAITING (2)

C2 priority ×2
A1 4 min | A2 5 min | A3 12 min
Unserved would cost 40

C3 priority ×1
A1 6 min | A2 4 min | A3 5 min
Unserved would cost 20

AVAILABLE UNITS (3)

A1 available
backs up A2

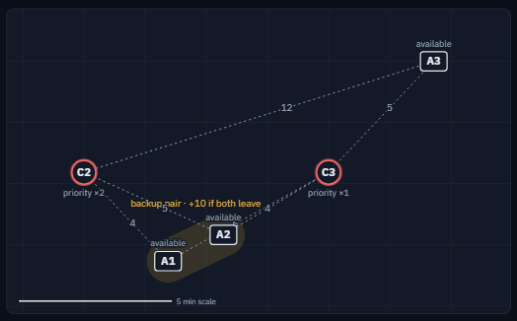
A2 available
backs up A1

A3 available
no backup link

SIMULATED RESPONSE ZONE

candidates only · no decision yet

▸ Animated Simulation | Reduced Motion



candidate route (dashed) · dispatched route · mutual backup pair (A1+A2)

Simulated response zone (synthetic mathematical topology). Euclidean distance equals travel time in minutes. Not a real GPS map or live road network. A4 is pre-assigned to critical call C1 and is outside this residual decision.

RUN A METHOD

- Nearest available
- Exact optimisation
- QAOA

Run dispatch analysis

No analysis has run.
Choose a method and run it. The assignment, its cost and its validation appear here.

RUNS THIS SESSION

Nothing has run on this scenario yet.

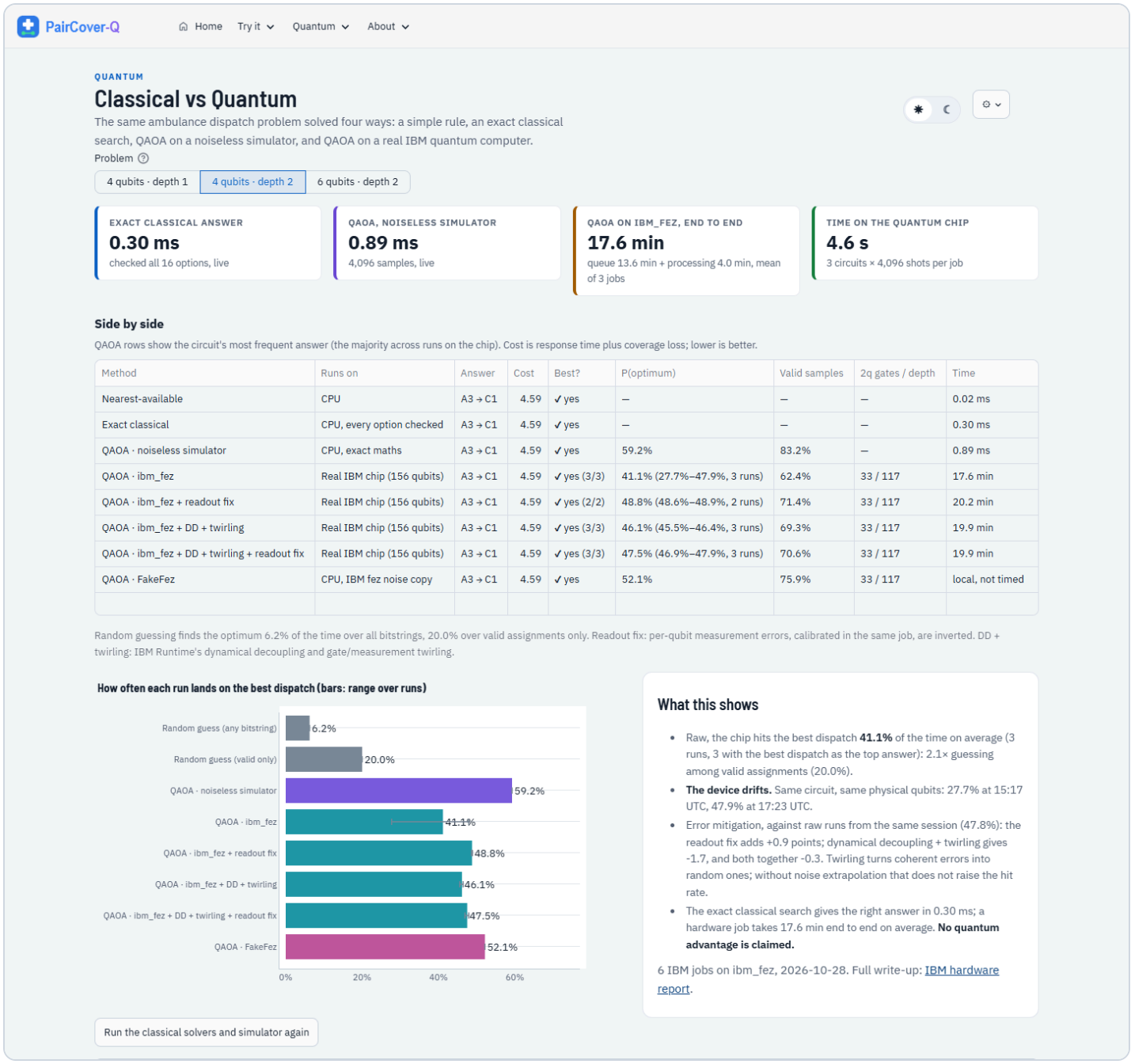
Open Try it, then Dispatch console. Click "Compare nearest vs exact". If time allows, open "Create your own emergency", click the map to place calls, and run it.

This is our dispatch console. On the left are the waiting calls and the available ambulances. In the middle is the response zone, where the dashed lines are travel minutes and the shaded pill is the backup pair A1 and A2.

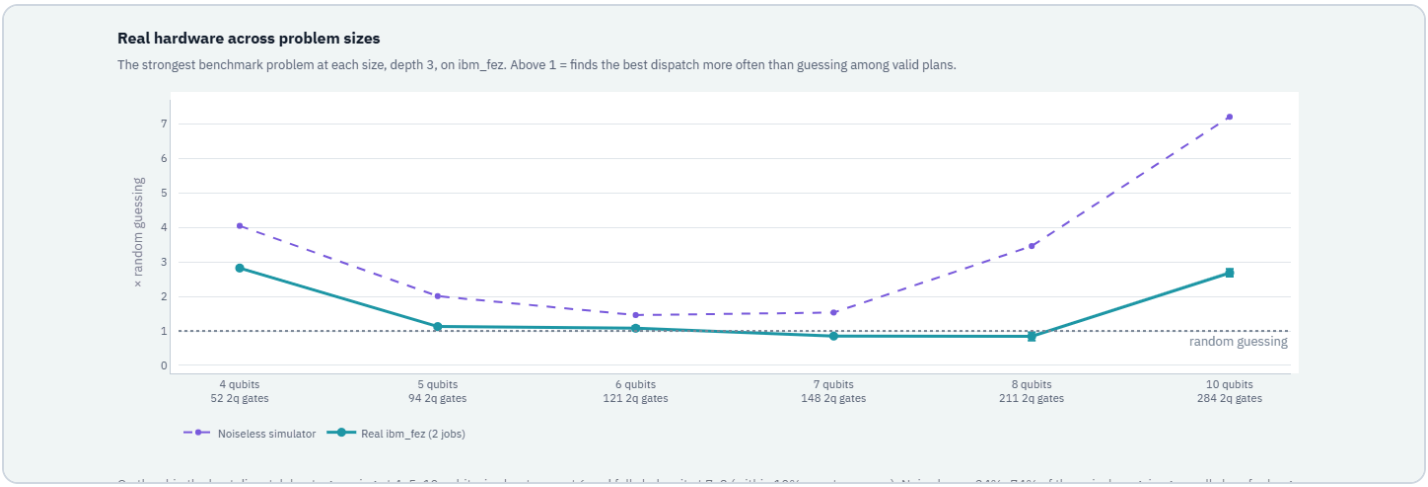
I'll run nearest-available and then the exact optimiser, and the console explains what changed: the exact plan keeps A2 home and the cost falls from 22 to 13.

---

4. Classical vs quantum, on a real IBM quantum computer (3 min)



Open Quantum, then Classical vs Quantum. Start on "4 qubits · depth 2". Point at the four timing cards, then the table, then the bar chart. Then click "6 qubits · depth 2" and show that the chip is only around guessing. Then scroll to the "Real hardware across problem sizes" chart.



Our mentor asked us to show classical and quantum side by side, and to run on real hardware. This page does both, on one dispatch problem.

The first two rows are classical: the nearest-ambulance rule and an exact search over every option. They run live, in well under a millisecond. The third row is QAOA on a noiseless simulator, also live. The next row is the same circuit run on **ibm_fez**, a real 156-qubit IBM quantum processor, where we ran it six times on 28 October: three times raw and three times with error mitigation. The last row is IBM's noise copy of the same chip.

On this 4-qubit problem, in all six runs the answer the real chip gave most often is the best dispatch, A3 to call 1, the same as the exact solver. In our evening session it hit the best plan in 46 to 49 percent of shots. Random guessing among valid plans gets 20 percent, so the chip is about 2.4 times better than guessing, and keeps about 80 percent of the noiseless 59 percent.

We also learned that the hardware drifts: our very first run, two hours earlier, on the same qubits, got only 28 percent. That is why we repeated every run and show the range.

We tried error mitigation. Correcting readout errors added about one point, because readout on this chip is already very good, about 1 percent error. Dynamical decoupling and twirling did not help. So the remaining error is from two-qubit gates, and that tells us what to work on next.

Now the latency. The classical answer takes a fraction of a millisecond. A quantum job took between 12 and 20 minutes end to end, mostly waiting in the queue. The circuits themselves ran on the chip for only 3 to 5 seconds per job. All six jobs used 45 seconds of our free 10 minutes a month.

> Now the 6-qubit problem. After mapping onto the chip the circuit has 82 two-qubit gates, and at that size the real chip finds the best plan between 5 and 10 percent of the time, around the 6.7 percent of guessing, and its most frequent answer is never the best plan. Notice also that here the nearest-ambulance rule picks a worse plan than the exact solver, because it ignores coverage. That is the point of our method.
>
> Finally, scroll down to "Real hardware across problem sizes". Following our mentor's advice to build on the hardware results, we ran the strongest problem at each size from 4 to 10 qubits on the chip. The chip beats guessing at 4 qubits, 2.8 times, and still at 10 qubits, 2.7 times, with the best plan as its top answer, but falls below guessing at 7 and 8. Noise keeps between a quarter and three quarters of the noiseless gain, so the chip only stays ahead where the algorithm's gain is large. The limit is hardware noise, not the algorithm.

---

5. Results, with honest limits (1.5 min)

ABOUT

Evidence

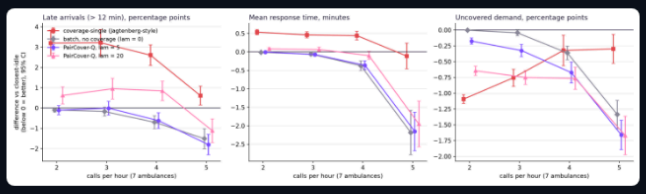
Does the idea hold up beyond one example? Two Round 2 studies: a simulated city over seven days, and QAOA on 23 problem instances generated by that simulation. No real ambulance records are used.

7-day simulated city | QAOA benchmark

Coverage-aware dispatch over time

- A 24 × 24 km synthetic region: 7 ambulances at 6 stations, 2 hospitals, a dense centre, a town and a sparse periphery.
- Calls arrive at random for **7 days**, repeated **30 times** per load; every policy sees the identical calls.
- With a small coverage weight (λ = 5), PairCover-Q showed **no statistically significant increase** in late arrivals or mean response time against nearest-available at any load, on both the main and held-out seeds. That is not proof of no harm: the intervals allow small effects either way.
- Uncovered demand fell at every load. Note this is the quantity the coverage term itself optimises, so it is expected rather than an independent outcome.
- At 5 calls/hour: mean response **−2.15 min** (95% CI −2.67 to −1.63), late arrivals **−1.80 points**. **Batch assignment without the coverage term gets −2.18 min**, so this gain comes from planning simultaneous calls together, not from the coverage term.
- Large coverage weights (λ ≥ 20) make arrivals later: protecting coverage has a price, and the simulation shows where it starts.

Full method and tables: rolling-simulation report.



Paired differences against nearest-available, 95% confidence intervals. Negative is better.

› How the coverage weight trades response time against coverage

› The synthetic region

Open About, then Evidence. Show the chart, then switch to the "QAOA benchmark" tab.

One example is not enough, so we ran a simulation: a synthetic 24 by 24 km city, 7 ambulances, calls arriving over 7 days, repeated 30 times at each load.

At 5 calls per hour, mean response time fell by 2.15 minutes compared with nearest-available. Most of this comes from planning simultaneous calls together: batching alone gives 2.18 minutes. The coverage term's own contribution is less uncovered demand, with no significant change in late arrivals.

For QAOA, we benchmarked 23 problems of 4 to 12 qubits. With a good starting strategy at depth 3, QAOA beat random guessing among valid plans on 19 of 23 problems in simulation. The hardware run you just saw shows how noise eats into that gain.

So we do not claim quantum advantage. A normal computer solves these in milliseconds. What we show is a complete quantum pipeline, from an emergency to a QUBO to a circuit on real IBM hardware and back to a dispatch decision, checked at every step against the exact answer.

---

6. Roadmap (1 min)

We now know where the hardware stops helping for our encoding: between 4 and 6 qubits, and that the limit is two-qubit gate noise, not readout. Our next steps attack exactly that.

1. **Zero-noise extrapolation** for two-qubit gate error. We already tried readout correction (+1 point) and dynamical decoupling with twirling (no gain).
2. **A feasibility-preserving XY mixer**, because on the real chip only 45 to 71 percent of samples were valid plans.
3. **Shallower circuits and angles tuned on the device**, instead of angles tuned only in simulation.
4. **Repeated hardware runs** on several days and devices: we saw a 20-point drift within two hours.

5. **Larger fleets**, where exact search stops being instant, which is where a quantum solver could start to matter.

If the judges ask...

- **"Is this faster than classical?"** No. Exact search takes under a millisecond; a hardware job took 12 to 20 minutes, mostly queue. We claim a validated pipeline and a measured path, not speed.
- **"Did you really run on hardware?"** Yes: ibm_fez, IBM Heron r2, 156 qubits, six jobs on 28 Oct 2026 (for example dbh0tjpt9uhs73c960mg), 4,096 shots per circuit. The raw counts are in reports/ibm_hardware in our repository.
- **"Did you try error mitigation?"** Yes. Readout correction added about 1 point; dynamical decoupling with gate and measurement twirling lowered the hit rate by about 2 points. The remaining error is two-qubit gate noise; zero-noise extrapolation is next.
- **"Why is the chip sometimes above the noiseless value?"** Most likely noise, not quantum gain: qubits relax towards 0, which favours plans with few ambulances sent, and our best plans are of that kind.
- **"Why does the real chip differ from the fake backend?"** The fake backend copies one calibration snapshot. The real chip drifts: our runs ranged from 28 to 48 percent while the copy gave 52, so we treat fake-backend numbers as a best case.
- **"How many qubits does a real problem need?"** One per eligible ambulance-call pair. Most real decisions are 2 qubits; 8 or more only when several calls arrive together.
- **"Why QAOA?"** The problem is naturally a QUBO, and QAOA is the standard hybrid algorithm for QUBOs on near-term devices.
- **"Is the data real?"** No, everything is synthetic and labelled that way. We have no real ambulance records.
- **"Why is the penalty set just above the bound?"** Bigger penalties make almost every sample valid but flatten the differences between plans, so the best one is harder to find. Our sweep showed 2.16x at the tight penalty against 0.62x at 4x.
- **"If you don't know":** "We have not tested that yet; it is on our roadmap."

Screens not captured here: Quantum Lab, Create your own emergency and Method & data. Show them live if asked.